

AntiFLipper: A Secure and Efficient Defense Against Label-Flipping Attacks in Federated Learning

Aashnan Rahman¹, Abid Hasan², Sherajul Arifin³, Faisal Haque Bappy⁴, Tahrim Hossain⁵,
Tariqul Islam⁶, Abu Raihan Mostofa Kamal⁷, and Md. Azam Hossain⁸

^{1,2,7,8} Islamic University of Technology, Bangladesh.

³ United International University, Bangladesh.

^{4,5,6} Syracuse University, United States.

Email: {aashnanrahman@iut-dhaka, abidhasan32@iut-dhaka, sherajul@cse.uiu.ac.bd, fbappy@syr, mhossa22@syr, mtislam@syr, azam@iut-dhaka, raihan.kamal@iut-dhaka}.edu

Abstract—Federated learning (FL) enables privacy-preserving model training by keeping data decentralized. However, it remains vulnerable to label-flipping attacks, where malicious clients manipulate labels to poison the global model. Despite their simplicity, these attacks can severely degrade model performance, and defending against them remains challenging. We introduce AntiFLipper, a novel and computationally efficient defense against multi-class label-flipping attacks in FL. The attacker’s capability is strictly limited to local data manipulation (label-flipping), without access to model updates or server components. Unlike existing methods that ensure security at the cost of high computational overhead, AntiFLipper employs a novel client-side evaluation strategy computing accuracy locally and sharing only a scalar score with the server, significantly reducing the central server’s burden during aggregation and making the approach suitable for edge networks. Comprehensive empirical evaluations across multiple datasets under different distributions demonstrate that AntiFLipper achieves accuracy comparable to state-of-the-art defenses while requiring substantially fewer computational resources on the server side. By balancing security and efficiency, AntiFLipper addresses a critical gap in existing defenses, making it particularly suitable for resource-constrained FL deployments (e.g., IoT) where both model integrity and operational efficiency are essential.

Index Terms—Federated Learning, Label Flipping Attack, Malicious Node Detection, Attack Mitigation, IoT, Edge Networks

I. Introduction

Federated learning (FL) has emerged as a prominent distributed learning paradigm for communication networks and edge computing systems, enabling geographically dispersed clients such as mobile devices, IoT sensors, and edge devices to collaboratively train a global model without sharing raw data [?], [?]. By exchanging only model updates, FL satisfies the strict privacy requirements of applications in healthcare, finance, and industrial IoT. However, the decentralized nature of FL exposes it to data poisoning attacks [?], [?], among which label-flipping attacks are particularly concerning for networked deployments. In this attack, malicious clients deliberately mislabel their local training data before submitting model updates. Critically, the attacker’s capability is strictly limited to this local label manipulation, they cannot modify model updates, intercept

communications, or access any server or peer data. Despite this simplicity, even a small fraction of compromised clients can cause substantial degradation of the global model [?], [?], posing a direct threat to the reliability of FL-enabled distributed systems [?].

Existing defenses typically rely on server-side anomaly detection or robust aggregation, requiring the server to inspect, cluster, or compare all client updates [?], [?], [?]. This introduces significant computational overhead at the aggregation server and, in some cases, requires the server to maintain a clean labeled reference dataset [?]¹—a privacy concern that contradicts FL’s data-minimization principles. Lightweight defenses, on the other hand, often fail under non-IID data distributions prevalent in heterogeneous wireless networks [?].

To address these limitations, we propose AntiFLipper, a defense that shifts detection to the client side through local accuracy evaluation, avoiding server-side model inspection entirely. Unlike approaches that depend on server-held reference data or expensive cryptographic mechanisms, AntiFLipper infers trust from relative client performance deviations, making it well-suited for resource-constrained edge and IoT deployments.

The core contributions of this paper are:

- **Attack Detection & Mitigation:** We propose AntiFLipper, a novel trust-based weighted aggregation method that detects and mitigates multi-class label-flipping attacks by identifying and excluding malicious clients, even in dynamic attack scenarios.
- **Accuracy and Efficiency:** AntiFLipper matches or exceeds state-of-the-art defenses in accuracy while achieving significantly lower aggregation time by avoiding server-side model update inspection.
- **Improved Scalability:** By shifting detection to client devices, AntiFLipper minimizes server burden, improving scalability in large-scale decentralized networks.

The remainder of the paper is structured as follows. In Section ??, we describe the problem statement and our research objectives. In Section ??, we review existing work related to the detection and defense against label-flipping attacks by malicious clients in federated learning scenarios.

In Section ??, we present AntiFLipper in detail. In Section ?? we present system implementation and performance evaluation. In Section ??, we provide a security analysis, discussing the robustness of our approach in mitigating malicious attacks. Finally, Section ?? summarizes the key findings and concludes the paper.

II. Problem Statement

A. Label Flipping Attack

Let $C = \{c_1, c_2, \dots, c_n\}$ be the set of clients participating in the federated learning process. Each client $c_i \in C$ possesses a local dataset $D_i = \{(x_k, y_k)\}_{k=1}^{m_i}$, where $x_k \in \mathcal{X}$ is the input feature and $y_k \in \mathcal{Y}$ is the true class label. In our threat model, the adversary is strictly limited to manipulating its local data (i.e., label-flipping) and has no capability beyond this scope. In a label-flipping attack, the adversary modifies this dataset into a corrupted version $D'_i = \{(x_k, y'_k)\}_{k=1}^{m_i}$, such that $y'_k \neq y_k$ for a substantial subset of the data [?]. These poisoned labels cause the local model m'_i , trained on D'_i , to encode incorrect decision boundaries. A common variant is the targeted label-flipping attack, where the adversary selects a source class $s \in \mathcal{Y}$ and a target class $\tau \in \mathcal{Y}$, applying:

$$y'_k = \begin{cases} \tau & \text{if } y_k = s, \\ y_k & \text{otherwise.} \end{cases}$$

The malicious client then trains locally and submits the resulting model m'_i to the server. Over time, the aggregation of poisoned updates subtly shifts the global model's decision boundaries toward the adversary's target class [?], relying solely on local label manipulation without requiring server access or deviating from the standard FL protocol.

B. Research Objectives

Existing defenses against label-flipping attacks either rely on centralized anomaly detection at the server, which is computationally expensive, unscalable, and burdensome for resource-constrained environments, or require clients to share portions of their local data, contradicting the privacy-preserving principles of FL.

To address these challenges, our research pursues the following three core objectives: i) design a decentralized aggregation method that accurately detects and mitigates label-flipping attacks; ii) maintain high model performance and reduce aggregation time even in the presence of adversarial clients, ensuring both robustness and efficiency; and iii) shift the majority of computations to the client side and rigorously analyze the trade-offs in system efficiency, scalability, and resilience under adversarial conditions. Together, these objectives aim to build a trustworthy federated learning framework that resists adversarial manipulation while preserving client privacy and enabling scalable deployment.

III. Related Work

A. Label Flipping-Specific Defenses

Several approaches have tackled label-flipping attacks in federated learning. Tolpegin et al. [?] demonstrated the feasibility of such attacks and proposed a Principal Component Analysis based mechanism to detect malicious clients, but their method lacks generalization guarantees and performs poorly in non-IID settings. Fung et al. [?] introduced FoolsGold, leveraging cosine similarity between gradient updates. While effective against sybil-style poisoning, it suffers from false positives in non-IID scenarios and incurs high computational cost.

Nguyen et al. [?] proposed FLAME, combining HDBSCAN clustering, differential privacy, and noise addition. It preserves model utility but requires careful tuning and introduces substantial overhead. Jebreel et al. [?] developed LFighter, reweighting client updates using feature-level centroid deviation. It improves robustness but its effectiveness declines with data heterogeneity.

Two recent works directly target label-flipping in non-IID FL settings and warrant closer examination. Li et al. [?] proposed an Honest Score Client Selection Scheme that evaluates clients against a clean server-held dataset. While effective in non-IID conditions, this approach requires the server to maintain labeled reference data, introducing a privacy concern as it contradicts the core FL principle that raw and sensitive client data should remain local. Shen et al. [?] introduced a privacy-preserving defense using cryptographic mechanisms. While offering stronger privacy guarantees, the method incurs significant computational overhead due to encryption and secure aggregation, raising scalability concerns for large client populations and resource-constrained edge devices. Notably, both works primarily validate their performance on simpler datasets such as MNIST, without demonstrating effectiveness on more complex datasets like CIFAR-10, raising concerns about generalizability.

B. Native Federated Learning Defenses

While not specifically tailored to counter label-flipping attacks, several robust aggregation methods have been studied in this context. FedAvg[?], the popular aggregation mechanism in federated learning, lacks robustness against adversarial updates and suffers significant accuracy degradation under targeted label-flipping. To improve resilience, Yin et al. [?] introduced coordinate-wise Median and Trimmed Mean strategies, demonstrating moderate robustness in IID settings but degraded performance in non-IID scenarios. Multi-Krum [?] uses distance-based filtering, assuming an IID setting and a known number of adversaries; its performance deteriorates with data heterogeneity and its quadratic complexity limits scalability.

In contrast, our proposed method, AntiFLipper, is designed without relying on data distribution assumptions or prior adversary knowledge. It provides a principled

aggregation framework that generalizes well across non-IID environments and varying model architectures. Notably, AntiFLipper significantly reduces computational overhead while maintaining high model accuracy across all evaluated scenarios, making it a practical and scalable solution for real-world federated learning applications where robustness and efficiency are critical.

IV. Detailed Design

AntiFLipper defends federated learning against label-flipping attacks via a lightweight, client-side accuracy evaluation mechanism. The core intuition is that a global model trained mostly on honest data will exhibit markedly lower accuracy on a client's poisoned data because flipped labels conflict with the learned representations. The client side mechanism independently evaluates the current global model on a subset of the client's private data and reports only a scalar accuracy to the server, transmitting neither raw data nor gradients and thus preserving privacy. From these scalars, the server maintains a per-client trust score, progressively down-weighting and eventually excluding clients with persistently anomalous accuracy. The method requires no explicit knowledge of the data distribution, it relies on relative performance deviations across clients, tolerating transient drops caused by natural data heterogeneity while penalizing consistently anomalous behaviour. Since attackers are restricted to label manipulation, they cannot affect the local evaluation process or any aspect beyond label corruption of training data. By deliberately avoiding server-side model-update inspection, the dominant overhead in competing methods, the framework keeps aggregation lightweight and scalable for resource-constrained edge and IoT deployments.

The AntiFLipper framework integrates five interdependent modular components as shown in Figure ?? . Each

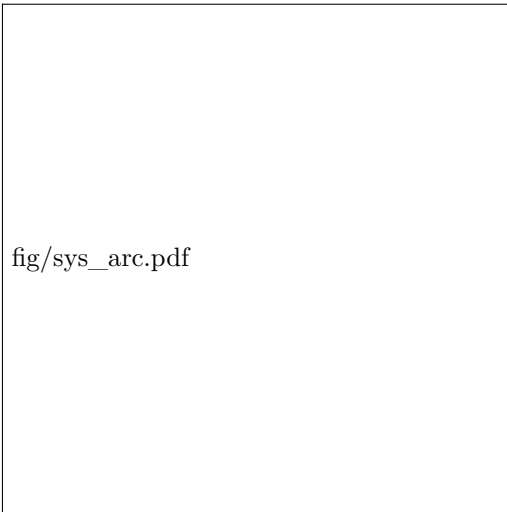


Fig. 1: Workflow of the AntiFLipper Framework

module operates within the standard FL loop, enhancing

Algorithm 1: AntiFLipper()

Input : Number of clients n , client list $clients$, local datasets $data$, learning rate α , trust threshold $\tau_{\text{threshold}}$, flag threshold cnt_{max}
Output: Global model M_{i+1} , malicious clients list β

```

1  $(M_0, \tau, cnt, \beta) \leftarrow \text{Initialize}()$ 
2 for  $i \leftarrow 1$  to  $totalRounds$  do
3   DistributeModel( $M_i, clients, \beta$ )
4   foreach  $j \in clients$  and  $j \notin \beta$  do
5      $(m_{ij}, acc_{ij}) \leftarrow \text{LocTrain}(M_i, j, data_{ij})$ 
6    $(\tau, cnt, \beta) \leftarrow \text{MalNodeFilter}(clients, acc_{ij}, \tau, cnt, \beta)$ 
7    $M_{i+1} \leftarrow \text{GlobModelAgg}(m, \tau, clients, \beta)$ 
8 return  $M_{i+1}, \beta$ 
```

robustness against label-flipping attacks without altering the fundamental decentralized [execution model](#).

A. M1: Local Evaluation Feedback Module

At the beginning of each communication round, the server distributes the global model M_i to all participating clients not in the malicious list β (Algorithm 1, lines 2–3). Each client-side evaluation mechanism then evaluates M_i on a subset of the client's local training dataset $data_{ij}$, computing a local accuracy score acc_{ij} . **Only this scalar is returned to the server, no raw data is transmitted.** Clients with poisoned data yield lower global model accuracy, as the model trained on honest data performs poorly on flipped labels, producing the detection signal. Following evaluation, each client performs local training (Algorithm 1, line 5) to generate model m_{ij} and returns both the model and acc_{ij} to the server.

B. M2: Trust Inference Module

Upon receiving the results, the server computes the average accuracy across all non-malicious clients (i.e., excluding clients identified as malicious up to the current round) (Algorithm ??, line 1). The trust score τ_j , initialized uniformly as $1/N$ where N is the total number of clients, is then updated for each client j based on the squared deviation from the computed average: $\tau_j \leftarrow \tau_j \pm \alpha \cdot (acc_{avg} - acc_{ij})^2$. Here, $\text{sqDeviation}() = (acc_{ij} - acc_{avg})^2$; $\text{penalizeTrust}() = \tau_j - \alpha \cdot d$; $\text{rewardTrust}() = \tau_j + \alpha \cdot d$; and $\text{normalizeTrust}()$ rescales trust into $[0, 1]$ excluding malicious clients. The sign is negative if $acc_{ij} < acc_{avg}$ (lines 4–5) and positive otherwise (lines 6–7). This quadratic penalty function penalizes deviations from the average more strongly than a linear scheme, addressing the issue where small but persistent differences could accumulate disproportionately over multiple rounds under a linear penalty formulation.

C. M3: Behavior Profiling Module

The server analyzes accuracy and trust scores over multiple rounds to build a behavior profile per client via the flag counter cnt_j (Algorithm 2, lines 8–11), tracking how

Algorithm 2: MalNodeFilter()

Input : Clients $clients$, Accuracies acc_{ij} , Trust values τ , Flags cnt , Malicious list β

Output: Updated τ , cnt , β

```
1  $acc_{avg} \leftarrow \text{getAvgAccuracy}(clients \setminus \beta)$ 
2 foreach  $j \in clients$  and  $j \notin \beta$  do
3    $d \leftarrow \text{sqDeviation}(acc_{ij}, acc_{avg})$ 
4   if  $acc_{ij} < acc_{avg}$  then
5      $\tau_j \leftarrow \text{penalizeTrust}(\tau_j, d, \alpha)$ 
6   else
7      $\tau_j \leftarrow \text{rewardTrust}(\tau_j, d, \alpha)$ 
8   if  $\tau_j \leq \tau_{\text{thresh}}$  then
9      $cnt_j \leftarrow cnt_j + 1$ 
10    if  $cnt_j \geq cnt_{\text{max}}$  then
11      add  $j$  to  $\beta$ 
12  $\tau \leftarrow \text{normalizeTrust}(\tau, \beta)$ 
13 return  $\tau$ ,  $cnt$ ,  $\beta$ 
```

many rounds a client's trust has fallen below threshold. AntiFlipper makes no assumptions about data distribution or non-IID structure and uses relative performance deviations across clients for detection. It tolerates occasional accuracy drops from natural heterogeneity, penalizing only persistent anomalies, enabling robust detection without distribution-aware mechanisms.

When malicious clients are few or their influence is negligible, they may not be immediately flagged, as their impact is insufficient to degrade the global model. Since the objective is to prevent meaningful model corruption, such low-impact cases are not critical to the defense goal.

D. M4: Anomaly-Guided Filtering Module

To avoid overreacting to transient accuracy drops, each client's flag counter cnt_j tracks underperformance rounds. A client with trust score τ_j below $\tau_{\text{thresh}} = \frac{1}{kN}$ (a scalar threshold inversely proportional to N , the total number of initial clients, and adjustable via k to control detection speed) is flagged (Algorithm ??, lines 8–9). After cnt_{max} flags, a client is deemed malicious and added to list β (lines 10–11). Flagged clients ($\tau_j < \tau_{\text{thresh}}$) have their updates excluded from the next round's aggregation to prevent model contamination. Malicious clients are excluded from participation in future rounds. Trust values are normalized to $[0, 1]$ (line 12). This balances tolerance for noisy data with protection against adversarial behavior.

The overall malicious node filtering process is implemented in Algorithm 2, which is invoked from the main AntiFlipper algorithm (Algorithm 1, line 6) during each communication round, before global model aggregation.

E. M5: Weighted Aggregation Module

After malicious clients have been filtered out, the remaining client models are aggregated using a weighted average (Algorithm 1, line 7). The weight assigned to

each client's model is proportional to its normalized trust score. Formally, the aggregated model M_{i+1} is computed as: $M_{i+1} = \sum_{j \in \text{Honest}} \frac{\tau_j}{W} \cdot m_{ij}$, where $W = \sum_{j \in \text{Honest}} \tau_j$. This ensures that reliable clients have a greater impact on the evolution of the global model, reinforcing the positive feedback loop while suppressing the influence of detected advisories.

V. Implementation and Evaluation

We developed AntiFlipper¹ in Python and evaluated it under label-flipping attacks against established defenses, including FedAvg [?], Median, Trimmed Mean [?], Fools-Gold [?], Tolpegin [?], MKRUM [?], FLAME [?], and LFighter [?]. Experiments were implemented in PyTorch on a 12th Gen Intel Core i9-12900K CPU, 128GB RAM, and NVIDIA RTX 3090 GPU running Ubuntu 22.04.4 LTS. We evaluated on two benchmark datasets: MNIST70K [?], comprising 70,000 grayscale digit images (60K train, 10K test), and CIFAR-10 [?], containing 60,000 color images across 10 classes (50K train, 10K test). MNIST models use a two-layer CNN followed by two fully connected layers, while CIFAR-10 models are based on ResNet18 [?] and ShuffleNetV2 [?], each ending with a single fully connected layer.

Experiments were conducted under both IID and non-IID settings. For MNIST, we simulated 100 clients with Dirichlet [?] non-IID partitioning ($\alpha = 1$), running for 200 communication rounds with 3 local epochs, batch size 64, and SGD (lr=0.001, momentum=0.9). For CIFAR-10, we used 100 clients (20 active per round) under IID and 20 clients under non-IID ($\alpha = 1$), trained over 100 rounds with 3 local epochs, batch size 32, and SGD (lr=0.01, momentum=0.9). All settings assume 40% malicious clients, the theoretical upper bound for MKRUM [?] performing constant label-flipping attacks.

A. Evaluation Metrics

We evaluated using three metrics: Overall Accuracy (All-Acc), Test Error (TE), and Aggregation Time. All-Acc measures prediction correctness; TE (from cross-entropy loss) reflects robustness against label-flipping; Aggregation Time measures server-side efficiency, essential for scalability.

B. Malicious Node Detection

AntiFlipper distinguishes malicious from honest clients using the global model's accuracy on local datasets. When a client flips labels, its local accuracy drops because the global model, trained on honest data, performs poorly on manipulated data. Malicious accuracy declines rapidly, near zero early on MNIST and within 20 rounds on CIFAR, while honest nodes maintain high accuracy. Figure ?? shows trust scores diverging clearly between honest and malicious clients within a few rounds.

¹The source code is available at: <https://github.com/abidh8820/AntiFlipper>

TABLE I: Comparison of Different Methods on MNIST and CIFAR-10 (IID and Non-IID)

	Methods	Fools-Gold	FedAvg	Median	TMean	FLAME	MKRUM	LFighter	Tolpegin	Anti-FLipper
MNIST (IID)	Global Accuracy (%)	0.36	51.85	96.10	96.18	97.44	97.45	97.45	97.45	97.06
	Test Error	17.60	0.86	0.18	0.18	0.09	0.09	0.09	0.09	0.10
	Aggregation Time (ms)	9.36	4.61	12.03	12.86	45.61	15.55	119.65	160.94	3.01
MNIST (NON-IID)	Global Accuracy (%)	3.58	53.99	95.67	95.61	97.03	96.74	96.76	96.80	96.77
	Test Error	6.72	0.76	0.22	0.23	0.11	0.12	0.13	0.13	0.11
	Aggregation Time (ms)	8.65	4.44	13.09	11.73	45.66	13.70	119.37	168.88	2.96
CIFAR (IID)	Global Accuracy (%)	70.13	47.21	63.90	63.77	67.30	32.86	69.96	70.28	70.09
	Test Error	1.10	1.43	1.10	1.12	1.15	2.31	1.12	1.11	1.10
	Aggregation Time (ms)	53.87	50.76	660.39	641.66	1489.91	686.93	468.02	354.96	48.01
CIFAR (NON-IID)	Global Accuracy (%)	56.01	50.95	60.93	59.66	37.77	44.08	64.58	53.58	68.23
	Test Error	1.12	1.31	1.18	1.19	3.23	1.70	1.21	1.31	1.14
	Aggregation Time (ms)	53.16	49.75	712.13	678.38	1334.06	727.38	449.12	339.47	48.43



Fig. 2: Average Client Trust Over First 25 Rounds

C. Experimental Results

On MNIST (Table ??), AntiFLipper, LFighter, Tolpegin, MKRUM, FLAME, TMean, and Median all achieved high accuracy. FoolsGold collapsed in non-IID settings (3.58%), misidentifying heterogeneous honest gradients as sybils. FedAvg amplified poisoned updates. AntiFLipper matched top defenses in accuracy while achieving the lowest aggregation time (3.01 ms IID; 2.96 ms non-IID), 53 times faster than LFighter and 15 times faster than FLAME.

On CIFAR-10, results followed similar trends with generally lower accuracy due to complexity. In IID, AntiFLipper (70.09%) matched LFighter, Tolpegin, and FoolsGold in terms of accuracy at the lowest aggregation time (48 ms vs. 1490 ms for FLAME). In non-IID, AntiFLipper led in both accuracy (68.23%) and efficiency, outperforming all methods including LFighter (64.58%) and Tolpegin (53.58%). **These CIFAR-10 non-IID results are particularly**

significant as they confirm that AntiFLipper’s relative-deviation approach generalizes to complex datasets and heterogeneous distributions without requiring distribution-specific assumptions or server-side reference data, in contrast to methods such as [?], [?] that were only validated on MNIST.

We further tested AntiFLipper under dynamic attack scenarios: (1) attacks starting after $n/2$ rounds (with n the total rounds), and (2) poisoning every n rounds ($n = 15$). As shown in Table ??, AntiFLipper maintained accuracies within $\pm 0.5\%$ of baseline and detected all malicious nodes. Delayed or intermittent attacks were easier to detect, as the model, already dominated by benign updates, remained stable and deviations were more apparent. In both cases, AntiFLipper remained robust, efficient, and incurred similar aggregation time as before.

TABLE II: Global test accuracies (%) under dynamic attack scenarios

Scenario	MNIST		CIFAR-10	
	IID	Non-IID	IID	Non-IID
Scenario 1	96.93	97.03	72.66	70.62
Scenario 2	96.68	96.82	72.59	70.59

In summary, AntiFLipper delivers robust, efficient, and scalable defense across datasets, distributions, and dynamic attack conditions.

VI. Security Analysis

In this section, we formalize AntiFLipper’s trust-based defense against label-flipping attacks. Our approach quantifies deviations in local model accuracy to adjust client trust scores, enabling the system to progressively isolate and exclude malicious participants. This establishes AntiFLipper as an efficient and robust defense for secure federated learning in adversarial federated learning environments.

A. Accuracy-Based Detection of Label Flipping Behavior

A key challenge in federated learning is mitigating label-flipping attacks, where malicious clients poison training by mislabeling data. AntiFLipper tackles this by exploiting the

accuracy drop observed when the global model is trained on such poisoned data.

Lemma-1: In AntiFLipper, if a client c_i flips labels in its local dataset d_i , converting it into a poisoned distribution d'_i , then the global model M yields lower accuracy on d'_i than on honest data, exposing adversarial behavior.

Proof: Let $C = \{c_1, c_2, \dots, c_n\}$ be the set of clients participating in federated learning, and let M be the global model aggregated from local models of the majority of honest clients. If a client $c_i \in C$ performs label-flipping, its local data distribution becomes d'_i , which diverges from the honest distributions d_j , where $j \neq i$. The global model M minimizes loss over honest client distributions d_j , so when evaluated on a flipped distribution d'_i , it incurs higher loss: $L(M, d'_i) > L(M, d_j)$. Since accuracy decreases as loss increases, this implies: $\text{acc}(M, d'_i) < \text{acc}(M, d_j)$. To detect deviation, AntiFLipper compares each client's accuracy to the average acc_{avg} . For a malicious client c_i , $\text{acc}_{\text{avg}} - \text{acc}(M, d'_i) > \text{acc}_{\text{avg}} - \text{acc}(M, d_j)$, and squaring amplifies this gap: $\Delta_i = (\text{acc}_{\text{avg}} - \text{acc}(M, d'_i))^2$. This deviation is used to reduce the client's trust score, allowing the system to gradually identify and isolate clients that consistently behave maliciously. [Transient non-IID deviations are handled by the flag-counter mechanism \(M4\), which requires persistent low trust before exclusion, preventing removal of honest clients with skewed data.](#)

B. Trust Score Convergence for Malicious Exclusion

Lemma-2: In AntiFLipper, a client c_i is considered malicious and excluded from training if its trust score satisfies $\tau_i < \tau_{\text{threshold}}$ for more than x rounds, indicating consistent behavior associated with label-flipping attacks.

Proof: Each client $c_i \in C$ starts with a trust value $\tau_i = \frac{1}{n}$, ensuring $\sum_{i=1}^n \tau_i = 1$. These values are used to weight local models in global aggregation as $M \leftarrow M + w_i \cdot m_i$, where $w_i = \frac{\tau_i}{W}$, $W = \sum \tau_i = 1$. After each round, trust is updated based on the squared deviation from the average accuracy: $\tau_i \leftarrow \tau_i \pm \alpha \cdot (\text{acc}_i - \text{acc}_{\text{avg}})^2$, and then normalized to maintain $\sum \tau_i = 1$. Since trust updates are applied independently per client, individual scores fluctuate while the total remains constant. A client is flagged if $\tau_i < \tau_{\text{threshold}} = 1/kn$, where $k > 1$, indicating its influence has dropped by a factor of k . Given $\tau_i \ll \tau_{\text{avg}} = 1/n$, its contribution becomes negligible. If this persists for x rounds, the client is considered malicious and excluded from further training.

VII. Computational Overhead of AntiFLipper

To quantify the computational overhead introduced by local evaluation in AntiFLipper, we consider a neural network model $f(x; w)$ with W parameters, trained on each client k with dataset D_k of n_k samples. Clients perform E local SGD epochs (batch size B), where the per-sample forward and backward pass costs are C_f and $C_b \approx 2C_f$,

respectively. Thus, the training cost per client per round is $C_{\text{train}} \approx 3E \cdot n_k \cdot C_f$.

Local evaluation on a fraction α of the data adds $C_{\text{eval}} = \alpha \cdot n_k \cdot C_f$, leading to a total cost of $C_{\text{total}} = n_k \cdot (3E + \alpha) \cdot C_f$, and relative overhead $\frac{\alpha}{3E}$. In our experiments with $E = 3$, full evaluation ($\alpha = 1$) yields a worst-case overhead of 11.1%. The global accuracies reported in Table ?? correspond to this full local data evaluation setting.

To reduce cost, we performed evaluation on only 10% of local data ($\alpha = 0.1$), resulting in just 1.1% overhead. Despite this, the system achieves strong performance: 97.11% (MNIST IID), 96.79% (MNIST Non-IID), 74.81% (CIFAR-10 IID), and 68.18% (CIFAR-10 Non-IID), with early malicious client detection, no increase in aggregation time, and no noticeable drop in accuracy—sometimes even slight improvements.

[AntiFLipper avoids server-side inspection of updates, eliminating the main overhead in competing methods. Instead of FLAME-style aggregation, it shifts evaluation to clients in parallel, keeping aggregation lightweight and scalable. The parameter \$\alpha\$ provides a flexible trade-off between efficiency and detection sensitivity.](#)

VIII. Conclusion

In this paper, we introduced AntiFLipper, a lightweight and effective defense mechanism for detecting and mitigating label-flipping attacks in federated learning. Unlike existing methods that trade off security for efficiency, AntiFLipper achieves a strong balance between both. Through extensive evaluations on IID and non-IID data distributions across two benchmark datasets, we show that AntiFLipper consistently matches or exceeds the accuracy of state-of-the-art defenses. Remarkably, it achieves significantly lower aggregation time—even outperforming the baseline FedAvg in some cases. This optimal trade-off makes AntiFLipper particularly suitable for real-world FL deployments in privacy-sensitive, time-constrained [edge and IoT network](#) environments where both security and efficiency are non-negotiable.